Analyse de la moyenne de température française (1950-2025)

Loïc Mémeteau Arthur Hauguel Margot Laignel

Objectif

Étude de la série temporelle mensuelle des températures en France, avec une décomposition additive :

- composante observée Y_t
- tendance T_t
- saisonnalité S_t
- résidu R_t

Chargement des données

```r
library(ggplot2)
library(forecast)

data <- read.csv("data/data.csv")
data <- data[, 2]
data <- ts(data, start = c(1950, 1), end = c(2025, 1), frequency = 12)

# Séparation temporelle train/test
train_data <- window(data, end = c(2023, 12))
test_data <- window(data, start = c(2024, 1))

head(train_data)
```

```
##            Jan       Feb       Mar       Apr       May       Jun
## 1950  1.953488  5.623044  6.662255  7.803161 13.850590 18.583679
```

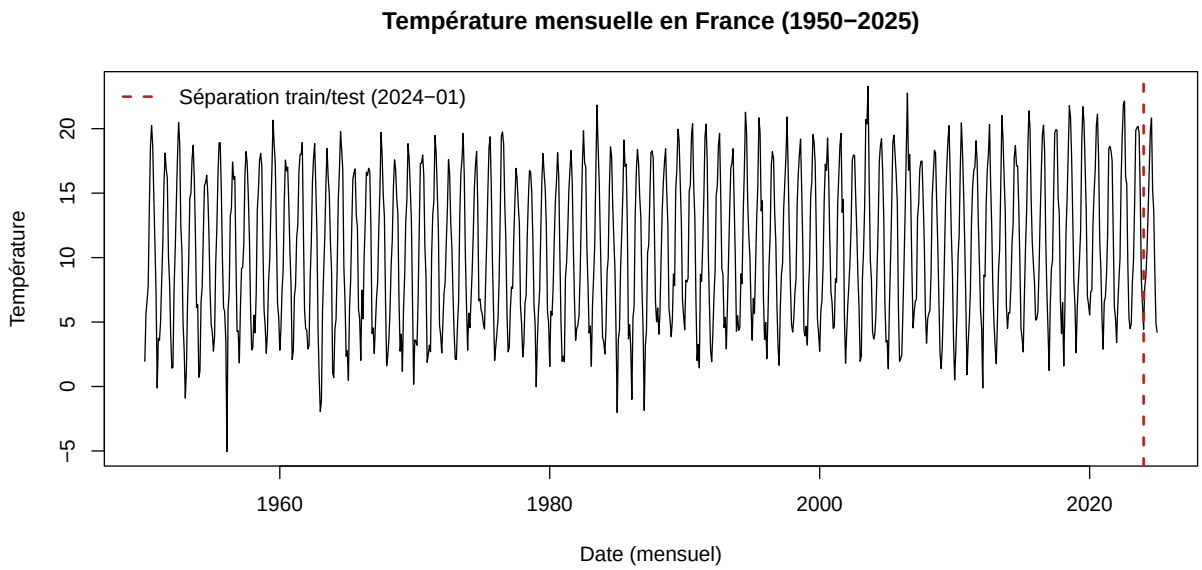
Les données sont mensuelles, de janvier 1950 à janvier 2025. La séparation est faite ainsi :

- **Train** : janvier 1950 à décembre 2023
- **Test** : janvier 2024 à janvier 2025

Visualisation initiale

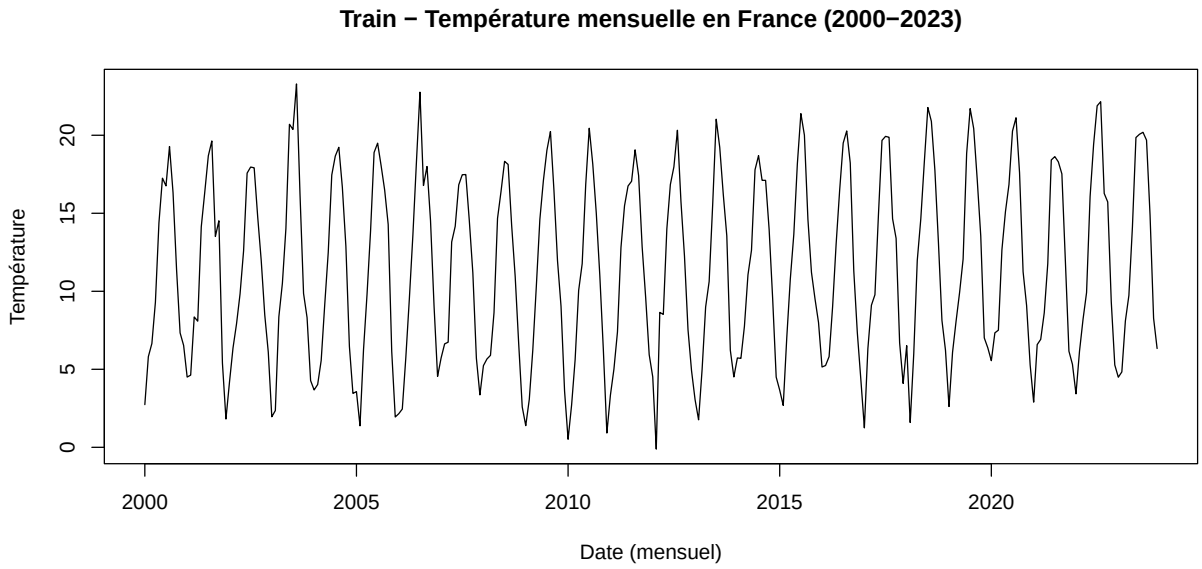
```r
plot.ts(
  data,
  main = "Température mensuelle en France (1950-2025)",
  ylab = "Température",
  xlab = "Date (mensuel)"
)
abline(v = 2024, col = "firebrick", lty = 2, lwd = 2)
legend(
  "topleft",
  legend = c("Séparation train/test (2024-01)"),
```

```
  col = c("firebrick"),
  lty = 2,
  lwd = 2,
  bty = "n"
)
```

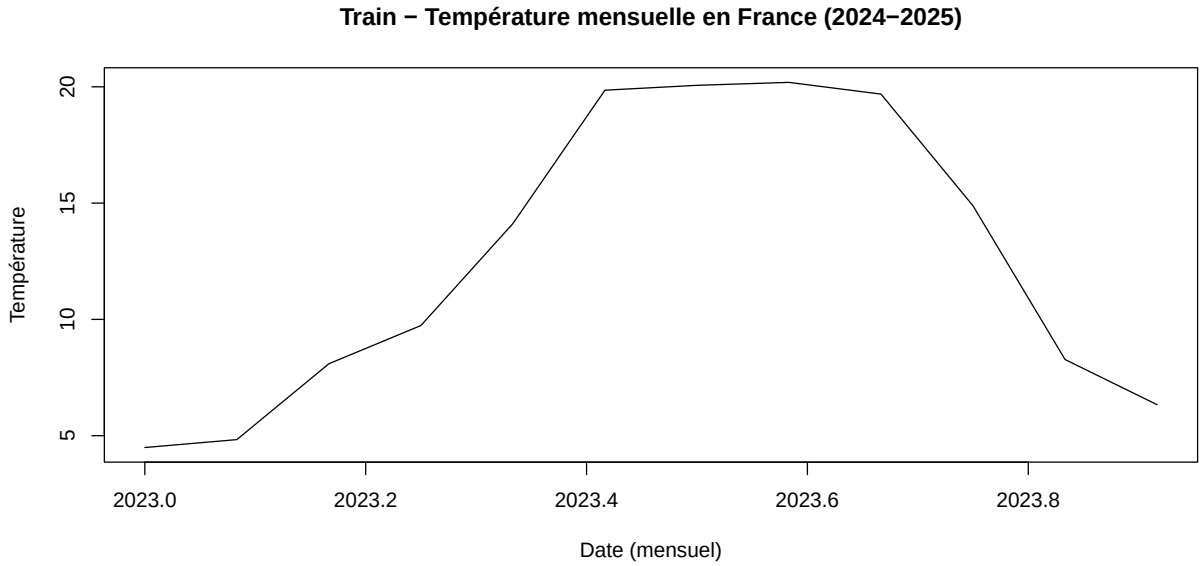


Observation : la série présente une saisonnalité visible et une tendance globale légèrement croissante.

```
short_data <- window(train_data, start = c(2000, 1))
plot.ts(
  short_data,
  main = "Train - Température mensuelle en France (2000-2023)",
  ylab = "Température",
  xlab = "Date (mensuel)"
)
```




```r
s_data <- window(train_data, start = c(2023, 1))
plot.ts(
  s_data,
  main = "Train - Température mensuelle en France (2024-2025)",
  ylab = "Température",
  xlab = "Date (mensuel)"
)
```



Remarque : les températures montent en général entre avril et août. Toute la décomposition ci-dessous est effectuée uniquement sur le **train**.

Décomposition additive et formules

Pour estimer la tendance, on utilise une moyenne mobile.

Moyenne mobile

$$\mathrm{MM}_t^{(m)} = \frac{1}{m} \sum_{j=-(m-1)/2}^{(m-1)/2} X_{t+j}$$

Pour $m = 12$ (ordre pair), la moyenne mobile centrée s'écrit :

$$\mathrm{MM}_{t,\text{centrée}}^{(12)} = \frac{1}{24} X_{t-6} + \frac{1}{12} \sum_{j=-5}^{5} X_{t+j} + \frac{1}{24} X_{t+6}$$

```
ma12 <- ma(short_data, 12)
ecart <- short_data - ma12
mois <- factor(cycle(short_data), levels = 1:12, labels = month.abb)
```

Modèle additif

$$Y_t = T_t + S_t + R_t$$

```
lm_temp <- lm(ecart ~ mois - 1, na.action = na.exclude)
coef_saison <- coef(lm_temp)
```

Pour imposer une somme saisonnière nulle :

$$\widetilde{S}_m = \widehat{S}_m - \frac{1}{12} \sum_{k=1}^{12} \widehat{S}_k$$

```
coef_saison <- coef_saison - mean(coef_saison, na.rm = TRUE)

saison_est <- ts(
  rep(coef_saison, length.out = length(short_data)),
  start = start(short_data),
  frequency = frequency(short_data)
)
```

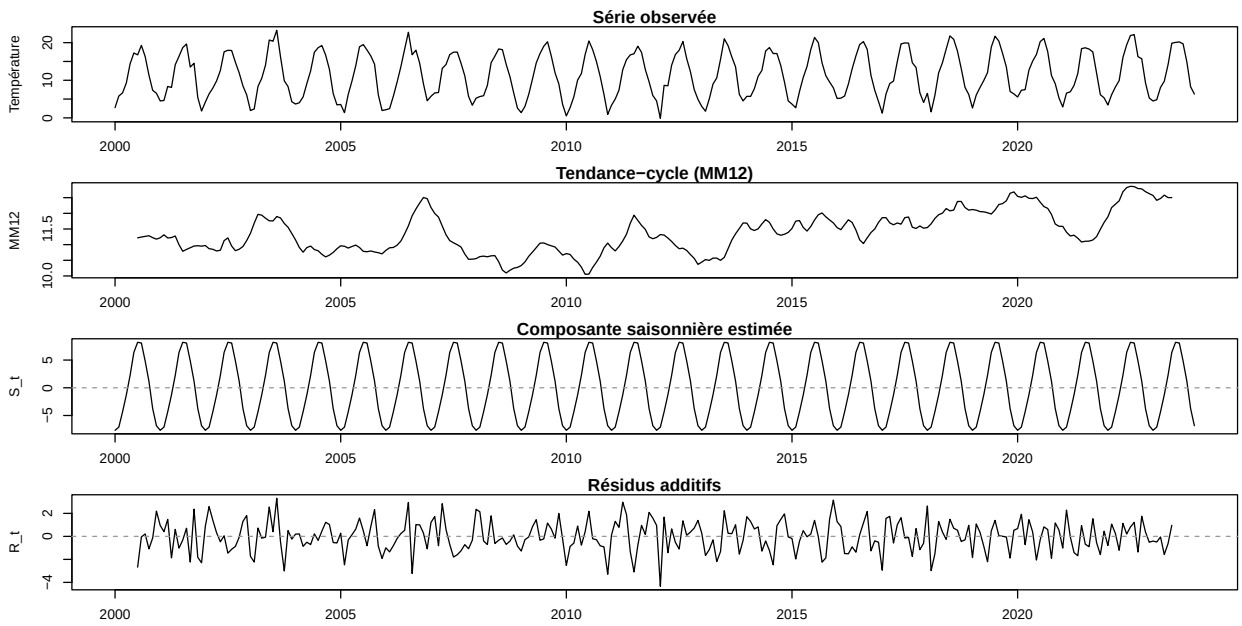
Le résidu additif est calculé par :

$$R_t = Y_t - (T_t + S_t)$$

```
residus <- short_data - (ma12 + saison_est)
```

Visualisation des composantes

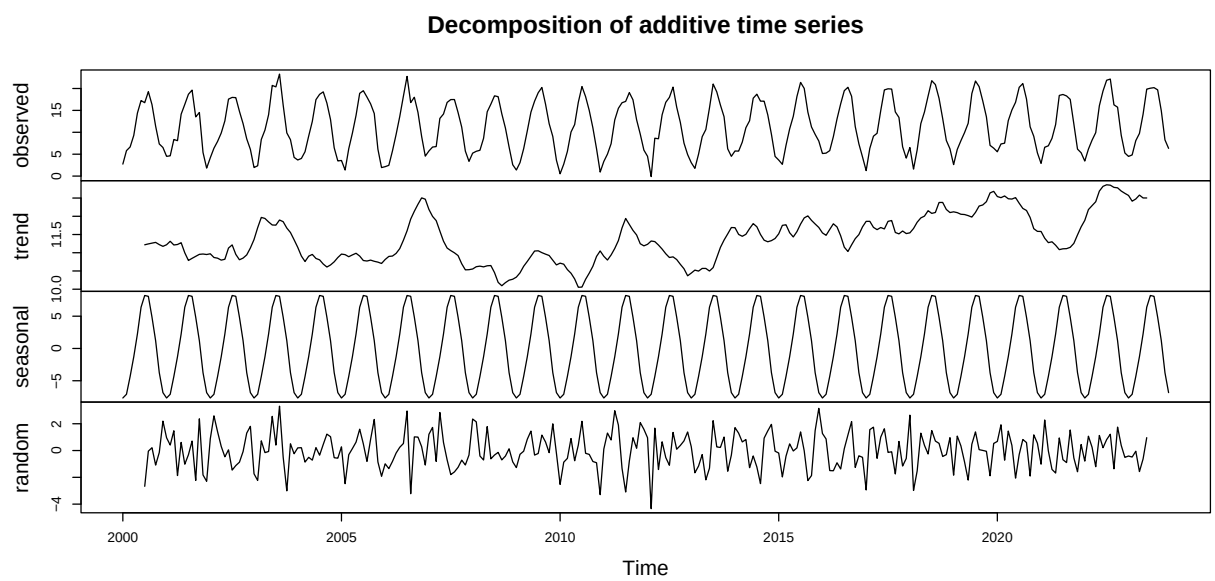
```
old_par <- par(no.readonly = TRUE)
par(mfrow = c(4, 1), mar = c(2.5, 4, 1.2, 1))

plot(short_data, main = "Série observée", ylab = "Température")
plot(ma12, main = "Tendance-cycle (MM12)", ylab = "MM12")
plot(saison_est, main = "Composante saisonnière estimée", ylab = "S_t")
abline(h = 0, col = "grey60", lty = 2)
plot(residus, main = "Résidus additifs", ylab = "R_t")
abline(h = 0, col = "grey60", lty = 2)
```




```r
par(old_par)
```

Nous pouvons avoir le même résultat avec `décompose`:

```r
m <- decompose(short_data, type = "additive")
plot(m)
```



Benchmark de la tendance polynomiale (série désaisonnalisée)

On définit :

$$y_t^{\text{cvs}} = y_t - \widehat{S}_t$$

puis on ajuste :

$$f_k(t) = \beta_0 + \beta_1 t + \cdots + \beta_k t^k, \quad k \in \{1, 2, 3\}$$

```r
decomp_train <- decompose(train_data, type = "additive")
y_cvs <- train_data - decomp_train$seasonal

df_poly <- na.omit(data.frame(
  t = seq_along(y_cvs),
  y = as.numeric(y_cvs)
))

poly_fits <- list(
  "Polynôme degré 1" = lm(y ~ t, data = df_poly),
  "Polynôme degré 2" = lm(y ~ t + I(t^2), data = df_poly),
  "Polynôme degré 3" = lm(y ~ t + I(t^2) + I(t^3), data = df_poly)
)

poly_cmp <- data.frame(
  Modele = names(poly_fits),
  R2_adj = sapply(poly_fits, function(mod) summary(mod)$adj.r.squared),
  AIC = sapply(poly_fits, AIC),
  BIC = sapply(poly_fits, BIC)
)
poly_cmp <- poly_cmp[order(poly_cmp$BIC), ]
knitr::kable(poly_cmp, digits = 4, caption = "Comparaison des tendances polynomiales")
```

Table 1: Comparaison des tendances polynomiales

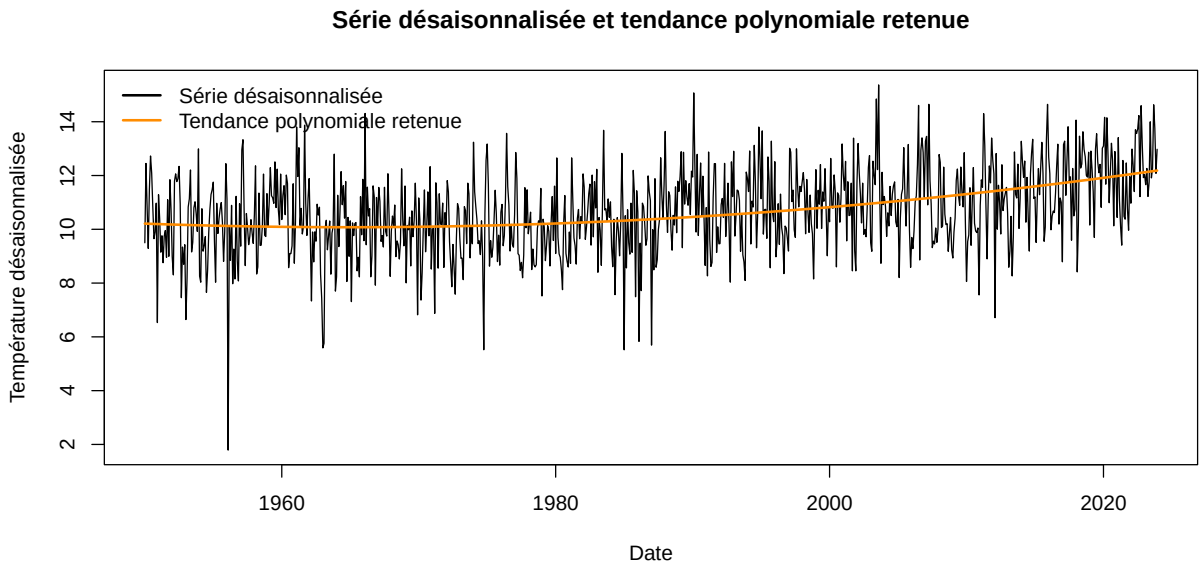
	Modele	R2_adj	AIC	BIC
Polynôme degré 2	Polynôme degré 2	0.1514	3200.190	3219.346
Polynôme degré 3	Polynôme degré 3	0.1505	3202.181	3226.126
Polynôme degré 1	Polynôme degré 1	0.1282	3223.184	3237.551

```r
best_poly_name <- as.character(poly_cmp$Modele[1])
best_poly_fit <- poly_fits[[best_poly_name]]
best_poly_name
```

```
## [1] "Polynôme degré 2"
```

```r
trend_poly <- ts(
  fitted(best_poly_fit),
  start = start(train_data),
  frequency = frequency(train_data)
)

plot(
  y_cvs,
  type = "l",
  main = "Série désaisonnalisée et tendance polynomiale retenue",
  ylab = "Température désaisonnalisée",
  xlab = "Date"
)
lines(trend_poly, col = "darkorange", lwd = 2)
```

```r
legend(
  "topleft",
  legend = c("Série désaisonnalisée", "Tendance polynomiale retenue"),
  col = c("black", "darkorange"),
  lty = 1,
  lwd = 2,
  bty = "n"
)
```




```r
res_poly <- residuals(best_poly_fit)

poly_diag <- data.frame(
  Modele = best_poly_name,
  LB_pvalue_24 = Box.test(
    res_poly,
    lag = 24,
    type = "Ljung-Box",
    fitdf = length(coef(best_poly_fit))
  )$p.value
)
knitr::kable(poly_diag, digits = 4, caption = "Blancheur des résidus de la tendance polynomiale retenue
```

Table 2: Blancheur des résidus de la tendance polynomiale retenue

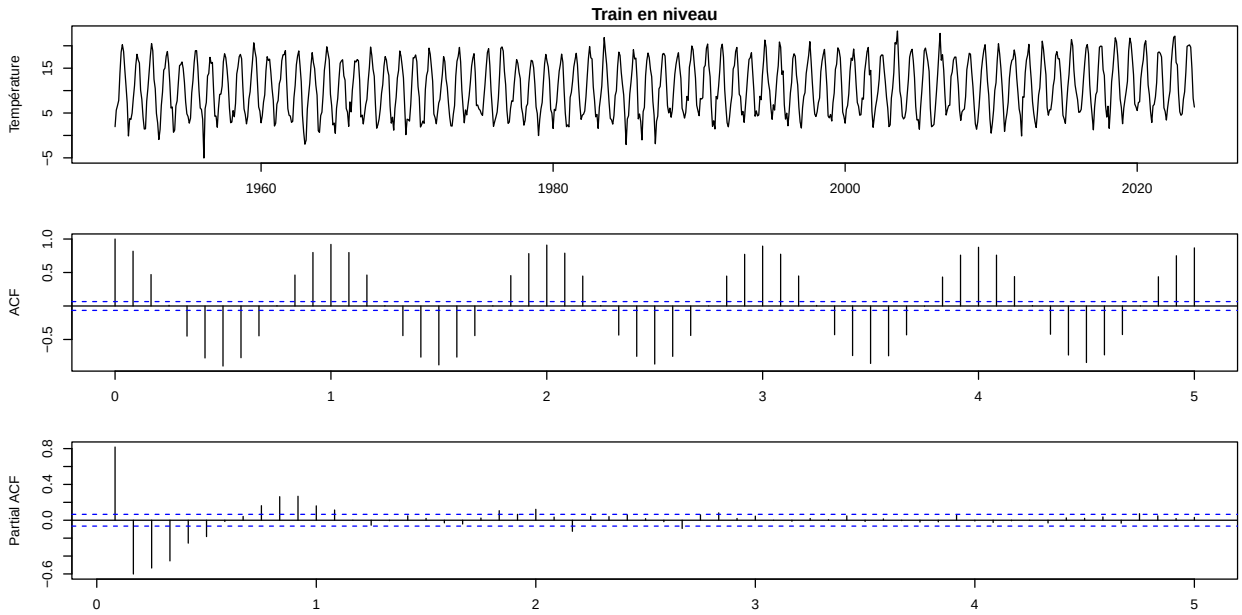
Modele	LB_pvalue_24
Polynôme degré 2	0.0222

Cette comparaison sert de **baseline de tendance** : Polynôme degré 2 est le meilleur parmi les polynômes testés. Mais ce n'est pas un modèle final de prévision : la p-value Ljung-Box vaut 0.0222, donc (< 0.05) les résidus ne sont pas blancs. Il reste une dépendance temporelle à modéliser, d'où l'étape SARIMA ensuite.

Identification Box-Jenkins (train)

Série en niveau : ACF/PACF

```
par(mfrow = c(3, 1), mar = c(2.8, 4, 1.5, 1))
plot(train_data, type = "l", main = "Train en niveau", ylab = "Température", xlab = "")
acf(train_data, lag.max = 60, main = "ACF(train)")
pacf(train_data, lag.max = 60, main = "PACF(train)")
```

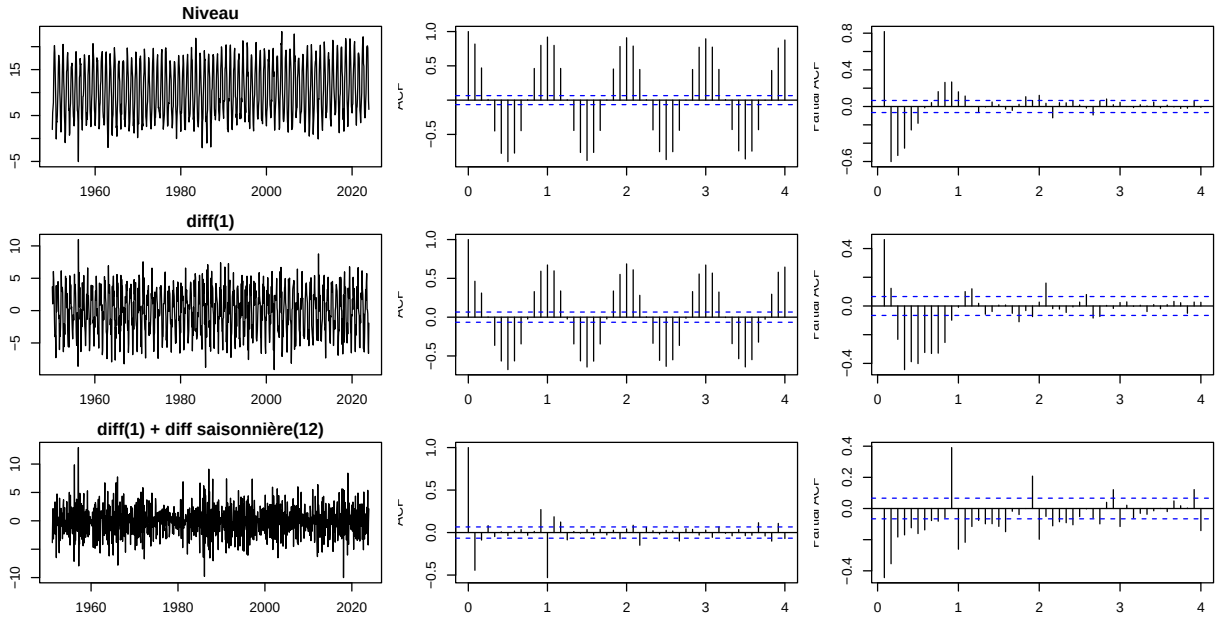



```
par(mfrow = c(1, 1))
```

La série en niveau présente une forte structure de dépendance et une saisonnalité marquée. On applique donc des différenciations avant d'ajuster des modèles ARMA/SARIMA.

Différenciations et stationnarisation

```
d1 <- diff(train_data, differences = 1)                # tendance
d12 <- diff(train_data, lag = 12, differences = 1)     # saisonnalité
z <- diff(d12, differences = 1)                        # combinaison d=1 et D=1

par(mfrow = c(3, 3), mar = c(2.5, 3.5, 1.6, 1))
plot(train_data, type = "l", main = "Niveau", ylab = "")
acf(train_data, lag.max = 48, main = "ACF niveau")
pacf(train_data, lag.max = 48, main = "PACF niveau")

plot(d1, type = "l", main = "diff(1)", ylab = "")
acf(d1, lag.max = 48, main = "ACF diff(1)")
pacf(d1, lag.max = 48, main = "PACF diff(1)")

plot(z, type = "l", main = "diff(1) + diff saisonnière(12)", ylab = "")
acf(z, lag.max = 48, main = "ACF z")
pacf(z, lag.max = 48, main = "PACF z")
```




```
par(mfrow = c(1, 1))
```

Tests de blancheur sur les séries transformées

```
lb_table <- data.frame(
  Serie = c("Niveau", "diff(1)", "diff(12)", "diff(1) + diff(12)"),
  p_value_LB_24 = c(
    Box.test(train_data, lag = 24, type = "Ljung-Box")$p.value,
    Box.test(d1, lag = 24, type = "Ljung-Box")$p.value,
    Box.test(d12, lag = 24, type = "Ljung-Box")$p.value,
    Box.test(z, lag = 24, type = "Ljung-Box")$p.value
  )
)
knitr::kable(lb_table, digits = 4, caption = "Tests de blancheur (Ljung-Box, lag 24)")
```

Table 3: Tests de blancheur (Ljung-Box, lag 24)

Serie	p_value_LB_24
Niveau	0
diff(1)	0
diff(12)	0
diff(1) + diff(12)	0

Même après stationnarisation, la série `z` n'est pas un bruit blanc : il reste une structure à modéliser.

Modèles candidats manuels (sur train)

```
score_fit <- function(fit, name, lag_lb = 24) {
  data.frame(
    Modele = name,
    AIC = AIC(fit),
```

```r
    BIC = BIC(fit),
    LB_pvalue = Box.test(
      residuals(fit),
      lag = lag_lb,
      type = "Ljung-Box",
      fitdf = length(coef(fit))
    )$p.value
  )
}

manual_fits <- list(
  "SARIMA(0,1,1)(0,1,1)[12]" = Arima(
    train_data, order = c(0, 1, 1),
    seasonal = list(order = c(0, 1, 1), period = 12),
    include.drift = FALSE, method = "ML"
  ),
  "SARIMA(1,1,1)(0,1,1)[12]" = Arima(
    train_data, order = c(1, 1, 1),
    seasonal = list(order = c(0, 1, 1), period = 12),
    include.drift = FALSE, method = "ML"
  ),
  "SARIMA(0,1,1)(1,1,1)[12]" = Arima(
    train_data, order = c(0, 1, 1),
    seasonal = list(order = c(1, 1, 1), period = 12),
    include.drift = FALSE, method = "ML"
  ),
  "SARIMA(2,1,0)(0,1,1)[12]" = Arima(
    train_data, order = c(2, 1, 0),
    seasonal = list(order = c(0, 1, 1), period = 12),
    include.drift = FALSE, method = "ML"
  ),
  "SARIMA(1,1,0)(0,1,1)[12]" = Arima(
    train_data, order = c(1, 1, 0),
    seasonal = list(order = c(0, 1, 1), period = 12),
    include.drift = FALSE, method = "ML"
  )
)

cmp_manual <- do.call(
  rbind,
  lapply(names(manual_fits), function(nm) score_fit(manual_fits[[nm]], nm))
)
cmp_manual <- cmp_manual[order(cmp_manual$AIC), ]
knitr::kable(cmp_manual, digits = 4, caption = "Comparaison des modèles manuels (train)")
```

Table 4: Comparaison des modèles manuels (train)

	Modele	AIC	BIC	LB_pvalue
2	SARIMA(1,1,1)(0,1,1)[12]	3215.611	3234.708	0.8217
1	SARIMA(0,1,1)(0,1,1)[12]	3228.200	3242.523	0.1562
3	SARIMA(0,1,1)(1,1,1)[12]	3228.204	3247.301	0.1824
4	SARIMA(2,1,0)(0,1,1)[12]	3392.533	3411.630	0.0000
5	SARIMA(1,1,0)(0,1,1)[12]	3495.501	3509.824	0.0000

```r
best_manual_name <- as.character(cmp_manual$Modele[1])
best_manual_fit <- manual_fits[[best_manual_name]]
best_manual_name
```

```
## [1] "SARIMA(1,1,1)(0,1,1)[12]"
```

```r
best_manual_fit
```

```
## Series: train_data
## ARIMA(1,1,1)(0,1,1)[12]
##
## Coefficients:
##          ar1      ma1     sma1
##       0.1337  -0.9765  -1.0000
## s.e.  0.0347   0.0090   0.0184
##
## sigma^2 = 2.151:  log likelihood = -1603.81
## AIC=3215.61   AICc=3215.66   BIC=3234.71
```

Modèle automatique

```r
fit_auto <- auto.arima(
  train_data,
  seasonal = TRUE,
  stepwise = FALSE,
  approximation = FALSE
)
fit_auto
```

```
## Series: train_data
## ARIMA(0,0,0)(1,1,1)[12]
##
## Coefficients:
##          sar1     sma1
##       -0.0466  -0.8810
## s.e.   0.0380   0.0192
##
## sigma^2 = 2.397:  log likelihood = -1634.32
## AIC=3274.64   AICc=3274.66   BIC=3288.96
```

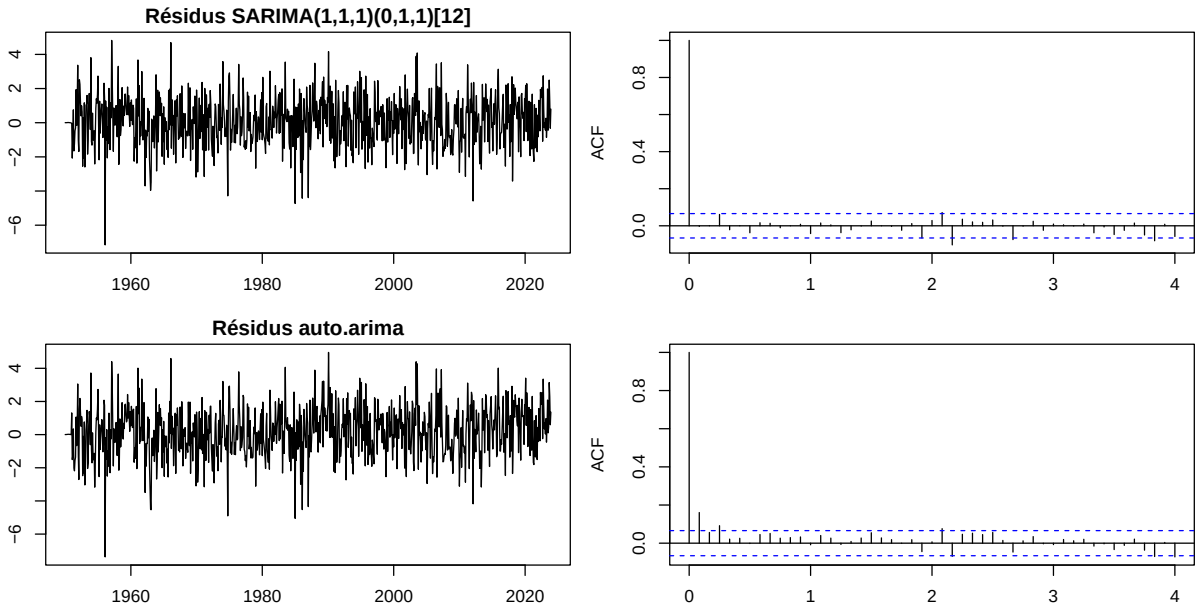
```r
cmp_auto <- rbind(
  score_fit(best_manual_fit, best_manual_name),
  score_fit(fit_auto, "auto.arima")
)
knitr::kable(cmp_auto, digits = 4, caption = "Meilleur modèle manuel vs auto.arima (train)")
```

Table 5: Meilleur modèle manuel vs auto.arima (train)

Modele	AIC	BIC	LB_pvalue
SARIMA(1,1,1)(0,1,1)[12]	3215.611	3234.708	0.8217
auto.arima	3274.637	3288.964	0.0007

Diagnostic des résidus (meilleur manuel vs auto)

```
par(mfrow = c(2, 2), mar = c(2.8, 3.8, 1.6, 1))
plot(residuals(best_manual_fit), type = "l", main = paste("Résidus", best_manual_name), ylab = "")
acf(residuals(best_manual_fit), lag.max = 48, main = "ACF résidus manuel")
plot(residuals(fit_auto), type = "l", main = "Résidus auto.arima", ylab = "")
acf(residuals(fit_auto), lag.max = 48, main = "ACF résidus auto")
```




```
par(mfrow = c(1, 1))

diag_res <- data.frame(
  Modele = c(best_manual_name, "auto.arima"),
  LB_pvalue_12 = c(
    Box.test(residuals(best_manual_fit), lag = 12, type = "Ljung-Box", fitdf = length(coef(best_manual_
    Box.test(residuals(fit_auto), lag = 12, type = "Ljung-Box", fitdf = length(coef(fit_auto)))$p.value
  ),
  LB_pvalue_24 = c(
    Box.test(residuals(best_manual_fit), lag = 24, type = "Ljung-Box", fitdf = length(coef(best_manual_
    Box.test(residuals(fit_auto), lag = 24, type = "Ljung-Box", fitdf = length(coef(fit_auto)))$p.value
  )
)
knitr::kable(diag_res, digits = 4, caption = "Blancheur des résidus des modèles retenus")
```

Table 6: Blancheur des résidus des modèles retenus

Modele	LB_pvalue_12	LB_pvalue_24
SARIMA(1,1,1)(0,1,1)[12]	0.6185	0.8217
auto.arima	0.0000	0.0007

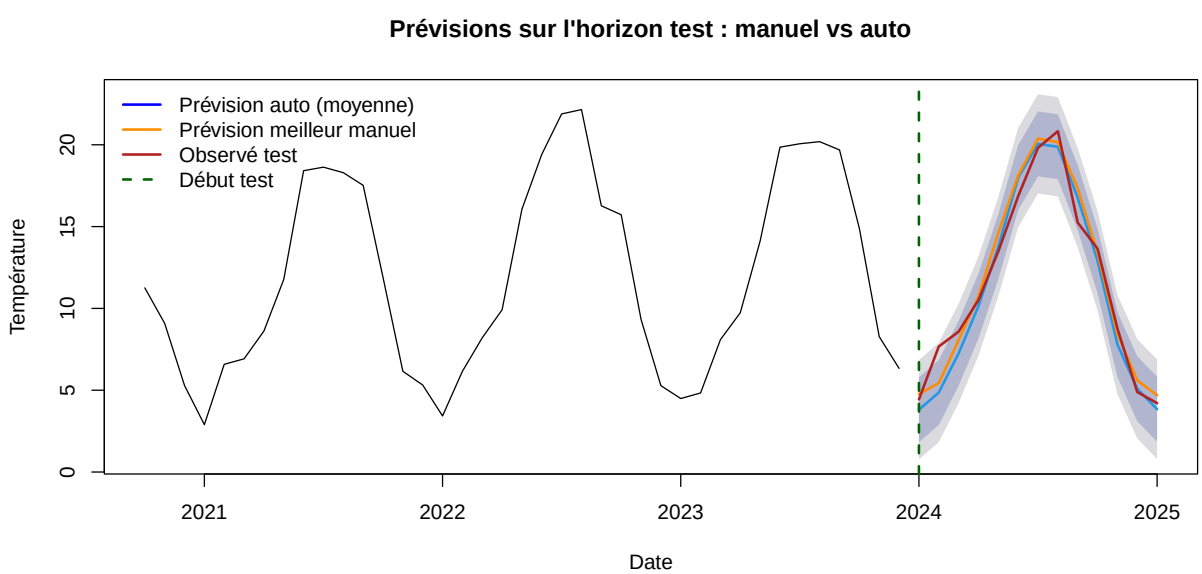
Prévision hors échantillon (train -> test)

Dans cette partie, on compare les prévisions du meilleur modèle manuel et du modèle automatique.

```r
h <- length(test_data)
n_train_window <- min(length(train_data), 3 * h)

fc_manual <- forecast(best_manual_fit, h = h)
fc_auto <- forecast(fit_auto, h = h)
```

Visualisation prévisions vs test

```r
plot(
  fc_auto,
  include = n_train_window,
  main = "Prévisions sur l'horizon test : manuel vs auto",
  ylab = "Température",
  xlab = "Date"
)
lines(fc_manual$mean, col = "darkorange", lwd = 2)
lines(test_data, col = "firebrick", lwd = 2)
abline(v = time(test_data)[1], col = "darkgreen", lty = 2, lwd = 2)
legend(
  "topleft",
  legend = c("Prévision auto (moyenne)", "Prévision meilleur manuel", "Observé test", "Début test"),
  col = c("blue", "darkorange", "firebrick", "darkgreen"),
  lty = c(1, 1, 1, 2),
  lwd = 2,
  bty = "n"
)
```



Évaluation sur le jeu de test

On calcule les métriques :

- $\text{RMSE} = \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}$
  Pénalise davantage les grosses erreurs que le MAE.

- MAE $= \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$
  Interprétation directe en unité de la variable (ici, °C).
- MAPE $= \frac{100}{n}\sum_{t=1}^{n}\left|\frac{y_t-\hat{y}_t}{y_t}\right|$
  Donne une erreur relative, plus facile à comparer entre séries/échelles.

```r
metric_fun <- function(y_true, y_pred) {
  rmse <- sqrt(mean((y_true - y_pred)^2))
  mae <- mean(abs(y_true - y_pred))
  idx_nz <- y_true != 0
  mape <- mean(abs((y_true[idx_nz] - y_pred[idx_nz]) / y_true[idx_nz])) * 100
  c(RMSE = rmse, MAE = mae, MAPE = mape)
}

y_true <- as.numeric(test_data)
y_pred_manual <- as.numeric(fc_manual$mean)
y_pred_auto <- as.numeric(fc_auto$mean)

met_manual <- metric_fun(y_true, y_pred_manual)
met_auto <- metric_fun(y_true, y_pred_auto)

metrics <- data.frame(
  Modele = c(best_manual_name, "auto.arima"),
  RMSE = c(met_manual["RMSE"], met_auto["RMSE"]),
  MAE = c(met_manual["MAE"], met_auto["MAE"]),
  MAPE_pct = c(met_manual["MAPE"], met_auto["MAPE"])
)
knitr::kable(metrics, digits = 3, caption = "Performance de prévision sur le jeu de test")
```

Table 7: Performance de prévision sur le jeu de test

Modele	RMSE	MAE	MAPE_pct
SARIMA(1,1,1)(0,1,1)[12]	1.050	0.832	8.627
auto.arima	1.128	0.890	9.535

Interprétation

- Les sections ci-dessus reprennent le pipeline vu en TP : identification, stationnarisation, choix de modèles, diagnostics résiduels et évaluation hors échantillon.
- Sur le jeu de test, SARIMA(1,1,1)(0,1,1)[12] est plus précis que `auto.arima` : RMSE 1.05 vs 1.128, MAE 0.832 vs 0.89, MAPE 8.627% vs 9.535%.
- Le modèle retenu pour la prévision finale est donc SARIMA(1,1,1)(0,1,1)[12].